2025

Java Behavioral Design Patterns

T2T
HIDHAS MOHAMED [25007]

1. **How each pattern improves flexibility and reusability**

Observer Pattern

The Observer pattern increases flexibility by decoupling the subject from its observers. The subject does not need to know the internal logic of the observers—it only notifies them when something changes. This allows new observers to be added or removed at runtime without modifying existing code. Reusability is also improved because the subject and observers can be reused independently in other projects, as they follow a common interface-based communication model.

Strategy Pattern

The Strategy pattern improves flexibility by allowing algorithms or behaviors to be selected at runtime. Instead of embedding logic inside conditional statements (if/else), each behavior is encapsulated inside its own class. This makes the code easier to modify or extend without breaking existing functionality. Reusability is increased because different strategies (e.g., payment types, sorting approaches, discount calculations) can be plugged into different parts of the system as needed.

Command Pattern

The Command pattern enhances flexibility by turning every action (such as turning on a light or turning off a fan) into an object. This decouples the sender of a request (Invoker) from the actual logic that performs the operation (Receiver). It also enables advanced features like undo/redo, logging, and macro commands without changing existing code. Because commands are independent classes, they can be reused easily across different invokers or devices.

2. **Which pattern was easiest/hardest to implement, and why.**

Easiest: Strategy Pattern

The Strategy pattern was the easiest to implement because its structure is straightforward, comprising one interface, several concrete strategies, and one context class. It is simple in flow and does not have many interacting components. The main idea of the pattern is about selecting which behavior to use, which conceptually makes it simpler.

Moderate: Command Pattern

The Command pattern was moderately difficult because it involved more components: the Command interface, concrete commands, receivers, and an invoker. To understand the flow, especially how the command object acts as a bridge between invoker and receiver, one needed to have a proper conceptual picture. Implementing the undo feature also added to the complexity.

Hardest: Observer Pattern

Observer was the most challenging because of its dynamic, multi-object relationship. Handling the list of observers, the automatic update of the observers, and avoiding tight coupling had to be carefully structured. Since many observers may react differently to the same event, deeper thinking was needed to understand how notifications would flow from subject to observer.

3. **Real-world examples where these patterns could apply**

Observer Pattern

UI Frameworks: Buttons signaling listeners about click events.

Stock Market Apps: When the price of a stock changes, notifies investors (observers).

Notification Systems: YouTube notified its subscribers.

Strategy Pattern

Payment Gateways : Choosing the type of payment to be used, such as Card, PayPal, Crypto.

Sorting Algorithms : quick sort, merge sort or bubble sort at runtime.

Game AI : Various movements or attack strategies for enemies.

Command Pattern b

Smart Home Automation: Remote-controlled lights, fans, switches-as implemented.

EDITORS Undo/Redo: Text editors store user actions as commands.

Task Scheduling: Command objects stored in job queues.